Nombres: Kendra Michelle.

Apellido: Carvajal Contreras.

Matricula: 2024-2572.

Materia: Programación Web.

Proyecto: Sistema de Gestión para Finanzas Personales.

Profesor: Luis Soto.

Sección: 2026-C-1.

Fecha: Sabado 25 de abril 2026.

Sistema de Gestión para Finanzas Personales.

Introducción:

En la actualidad, la gestión de las finanzas personales se ha convertido en una necesidad fundamental para mantener un control adecuado de los ingresos y gastos. Muchas personas no cuentan con herramientas eficientes para registrar sus movimientos económicos, lo que dificulta la toma de decisiones financieras. Por esta razón, el desarrollo de aplicaciones que permitan llevar un control organizado y automatizado resulta de gran utilidad.

El presente proyecto consiste en el desarrollo de una aplicación web para el control de gastos personales, la cual permite registrar, visualizar, actualizar y eliminar transacciones financieras. La aplicación está diseñada con una arquitectura cliente-servidor, utilizando tecnologías modernas como JavaScript, Node.js, Express y MongoDB, lo que garantiza un sistema dinámico, escalable y funcional.

A través de esta solución, el usuario puede obtener un resumen claro de su situación financiera mediante indicadores como ingresos, gastos y balance total. Además, se incorporan funcionalidades adicionales como filtros y validaciones que mejoran la experiencia de uso y la precisión de la información almacenada.

Objetivo:

Desarrollar una aplicación web que permita gestionar de manera eficiente los gastos personales, facilitando el registro, consulta, actualización y eliminación de transacciones, así como el análisis de la información financiera mediante indicadores claros y organizados.

Descripción del proyecto:

El proyecto consiste en una aplicación web de control de gastos personales basada en una arquitectura cliente-servidor. El sistema está compuesto por un frontend desarrollado en HTML, CSS y JavaScript, y un backend construido con Node.js y Express, conectado a una base de datos NoSQL en MongoDB Atlas.

La aplicación permite a los usuarios registrar transacciones indicando descripción, monto, tipo (ingreso o gasto), categoría y fecha. Estas transacciones son almacenadas en la base de datos y pueden ser consultadas en una tabla dinámica dentro de la interfaz.

Además, el sistema calcula automáticamente indicadores financieros como el total de ingresos, total de gastos y el balance disponible. También se incluyen funcionalidades de filtrado por categoría y tipo, así como opciones para editar y eliminar registros existentes.

Requisitos técnicos implementados:

- Desarrollo de una API REST utilizando Node.js y Express.
- Conexión a base de datos NoSQL mediante MongoDB Atlas.
- Implementación de operaciones CRUD:
  - Crear transacciones.
  - Leer transacciones.
  - Actualizar transacciones.
  - Eliminar transacciones
- Uso de Mongoose para la definición de esquemas y modelos.
- Consumo de API desde el frontend mediante `fetch`.
- Manejo de datos en formato JSON.
- Arquitectura cliente-servidor.

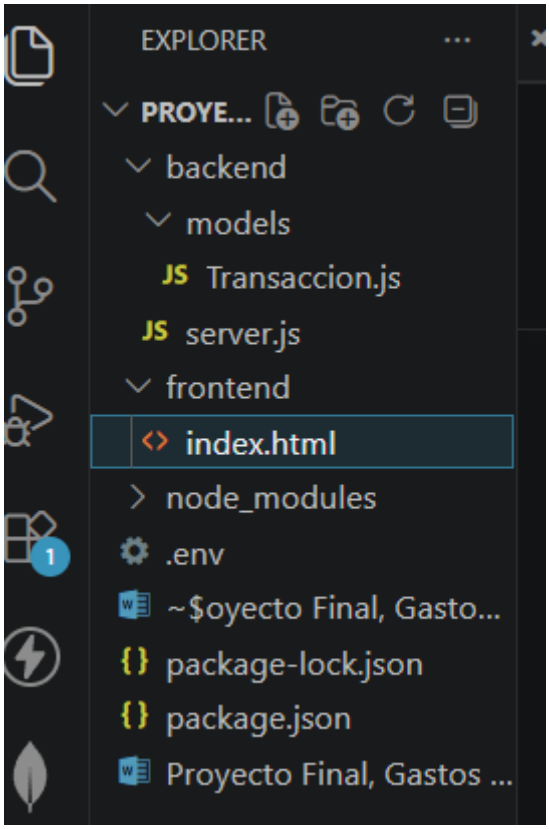
- Implementación de rutas REST:
  - POST /api/transacciones.
  - GET /api/transacciones.
  - PUT /api/transacciones/:id.
  - DELETE /api/transacciones/:id

Validaciones implementadas:

- Verificación de campos obligatorios (descripción, monto, tipo, categoría y fecha).
- Validación de longitud mínima en:
  - Descripción (mínimo 3 caracteres).
  - Categoría (mínimo 2 caracteres)
- Validación de monto:
  - Debe ser mayor a 0
- Confirmación antes de:
  - Actualizar una transacción.
  - Eliminar una transacción.
- Manejo de errores en backend mediante try-catch.
- Respuestas HTTP adecuadas:
  - 201 Created.
  - 400 Bad Request.
  - 404 Not Found.
  - 500 Internal Server Error
- Mensajes visuales en el frontend para informar al usuario.

Estructura del proyecto:

El proyecto se organizó de la siguiente manera:



Funcionamiento de las rutas:

- **GET /api/transacciones:**

  Obtiene la lista completa de transacciones almacenadas en la base de datos MongoDB.

- **GET /api/transacciones/:id:**

  Permite obtener una transacción específica mediante su ID.

  Si la transacción no existe, devuelve un error **404**.

- **POST /api/transacciones:**

  Permite crear una nueva transacción (ingreso o gasto).

  Se validan los datos antes de guardarlos en la base de datos.

  Devuelve código **201** si se crea correctamente.

- **PUT /api/transacciones/:id:**

  Permite actualizar los datos de una transacción existente.

  Si la transacción no existe, devuelve error **404**.

- **DELETE /api/transacciones/:id:**

  Elimina una transacción según su ID.

  Si no existe, devuelve error **404**.

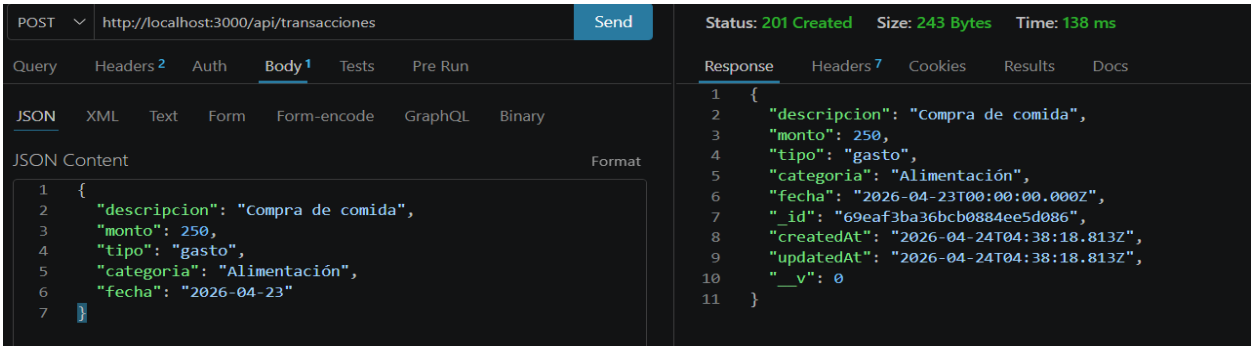
Herramientas utilizadas:

- **Visual Studio Code** → Editor de código.
- **Node.js** → Entorno de ejecución para JavaScript.
- **Express.js** → Framework para el backend.
- **MongoDB Atlas** → Base de datos en la nube.
- **Mongoose** → ODM para MongoDB.
- **JavaScript (Vanilla)** → Lógica del frontend.
- **HTML5** → Estructura de la interfaz.
- **CSS3** → Estilos y diseño.
- **Thunder Client / Postman** → Pruebas de API.
- **Git (opcional)** → Control de versiones.

Pruebas de funcionamiento:

Se realizaron pruebas utilizando Postman para verificar el correcto funcionamiento de cada endpoint:

- **Creación de usuarios (POST):**



- **Consulta de usuarios (GET):**

GET http://localhost:3000/api/transacciones Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2    "descripcion": "Compra de comida",
3    "monto": 250,
4    "tipo": "gasto",
5    "categoria": "Alimentación",
6    "fecha": "2026-04-23"
7  }
```

Status: 200 OK Size: 245 Bytes Time: 73 ms

Response Headers 7 Cookies Results Docs

```
1  [
2    {
3      "_id": "69eaf3ba36bcb0884ee5d086",
4      "descripcion": "Compra de comida",
5      "monto": 250,
6      "tipo": "gasto",
7      "categoria": "Alimentación",
8      "fecha": "2026-04-23T00:00:00.000Z",
9      "createdAt": "2026-04-24T04:38:18.813Z",
10     "updatedAt": "2026-04-24T04:38:18.813Z",
11     "__v": 0
12   }
13 ]
```

- **Actualización de usuarios (PUT):**

PUT http://localhost:3000/api/transacciones/69eaf3ba36bcb0884ee5d086 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2    "descripcion": "Compra actualizada",
3    "monto": 300,
4    "tipo": "gasto",
5    "categoria": "Alimentación",
6    "fecha": "2026-04-23"
7  }
```

Status: 200 OK Size: 245 Bytes Time: 83 ms

Response Headers 7 Cookies Results Docs

```
1  {
2    "_id": "69eaf3ba36bcb0884ee5d086",
3    "descripcion": "Compra actualizada",
4    "monto": 300,
5    "tipo": "gasto",
6    "categoria": "Alimentación",
7    "fecha": "2026-04-23T00:00:00.000Z",
8    "createdAt": "2026-04-24T04:38:18.813Z",
9    "updatedAt": "2026-04-24T04:47:04.975Z",
10   "__v": 0
11 }
```

- **Eliminación de usuarios (DELETE):**

DELETE http://localhost:3000/api/transacciones/69eaf3ba36bcb0884ee Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2    "descripcion": "Compra actualizada",
3    "monto": 300,
4    "tipo": "gasto",
5    "categoria": "Alimentación",
6    "fecha": "2026-04-23"
7  }
```

Status: 200 OK Size: 50 Bytes Time: 60 ms

Response Headers 7 Cookies Results Docs

```
1  {
2    "mensaje": "Transacción eliminada correctamente"
3  }
```

Código Fuente:

- **Backend/Models/Transaccion.js/server.js:**

backend > models > JS Transaccion.js > ...

```javascript
const mongoose = require('mongoose');

const transaccionSchema = new mongoose.Schema({
  descripcion: {
    type: String,
    required: true,
    trim: true
  },
  monto: {
    type: Number,
    required: true
  },
  tipo: {
    type: String,
    required: true,
    enum: ['ingreso', 'gasto']
  },
  categoria: {
    type: String,
    required: true
  },
  fecha: {
    type: Date,
    required: true
  }
}, {
  timestamps: true
});

module.exports = mongoose.model('Transaccion', transaccionSchema);
```

backend > JS server.js > ...

```javascript
require('dotenv').config();

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const Transaccion = require('./models/Transaccion');

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect(process.env.MONGO_URI)
  .then(() => {
    console.log('MongoDB conectado');

    app.listen(process.env.PORT || 3000, () => {
      console.log(`Servidor en http://localhost:${process.env.PORT || 3000}`);
    });
  })
  .catch((err) => {
    console.log('Error conectando a MongoDB:', err.message);
  });

app.get('/', (req, res) => {
  res.send('API funcionando');
});

app.post('/api/transacciones', async (req, res) => {
  try {
    const nuevaTransaccion = new Transaccion(req.body);
    const transaccionGuardada = await nuevaTransaccion.save();
    res.status(201).json(transaccionGuardada);
```

backend > JS server.js > ...

```
app.post('/api/transacciones', async (req, res) => {
    } catch (error) {
      res.status(400).json({ error: error.message });
    }
  });

  app.get('/api/transacciones', async (req, res) => {
    try {
      const transacciones = await Transaccion.find().sort({ fecha: -1 });
      res.json(transacciones);
    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  });

  app.get('/api/transacciones/:id', async (req, res) => {
    try {
      const transaccion = await Transaccion.findById(req.params.id);

      if (!transaccion) {
        return res.status(404).json({ mensaje: 'Transacción no encontrada' });
      }

      res.json(transaccion);
    } catch (error) {
      res.status(500).json({ error: error.message });
    }
  });

  app.put('/api/transacciones/:id', async (req, res) => {
    try {
      const transaccionActualizada = await Transaccion.findByIdAndUpdate(
        req.params.id,
```

```
65          req.params.id,
66          req.body,
67          { new: true, runValidators: true }
68        );
69
70        if (!transaccionActualizada) {
71          return res.status(404).json({ mensaje: 'Transacción no encontrada' });
72        }
73
74        res.json(transaccionActualizada);
75      } catch (error) {
76        res.status(400).json({ error: error.message });
77      }
78    });
79
80    app.delete('/api/transacciones/:id', async (req, res) => {
81      try {
82        const transaccionEliminada = await Transaccion.findByIdAndDelete(req.params.id);
83
84        if (!transaccionEliminada) {
85          return res.status(404).json({ mensaje: 'Transacción no encontrada' });
86        }
87
88        res.json({ mensaje: 'Transacción eliminada correctamente' });
89      } catch (error) {
90        res.status(500).json({ error: error.message });
91      }
92    });
```

- **Frontend/Index,html:**

```
frontend > index.html > html > body > script > limpiarFormulario
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Control de Gastos Personales</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f4f6f8;
      margin: 0;
      padding: 20px;
    }

    h1 {
      text-align: center;
      font-size: 40px;
    }

    .resumen, form, table {
      background: white;
      padding: 25px;
      margin: 25px auto;
      max-width: 1100px;
      border-radius: 10px;
      box-shadow: 0 0 12px #ccc;
    }

    .resumen {
      display: flex;
      justify-content: space-around;
      text-align: center;
    }
```

```
}

.resumen h3 {
  font-size: 24px;
}

.resumen p {
  font-size: 22px;
  font-weight: bold;
}

input, select, button {
  padding: 12px;
  margin: 8px;
  font-size: 16px;
}

button {
  cursor: pointer;
  border: none;
  border-radius: 6px;
}

.guardar {
  background: #2ecc71;
  color: white;
}
```

```
49  button {
50    cursor: pointer;
51    border: none;
52    border-radius: 6px;
53  }
54
55  .guardar {
56    background: #2ecc71;
57    color: white;
58  }
59
60  .cancelar {
61    background: #eee;
62    color: black;
63  }
64
65  .editar {
66    background: #3498db;
67    color: white;
68  }
69
70  .eliminar {
71    background: #e74c3c;
72    color: white;
73  }
74
75  table {
76    width: 100%;
77    border-collapse: collapse;
78    padding: 0;
```

```
    width: 100%;
    border-collapse: collapse;
    padding: 0;
    overflow: hidden;
  }

  th, td {
    border-bottom: 1px solid #ddd;
    padding: 15px;
    text-align: center;
    font-size: 18px;
  }

  th {
    background: #fafafa;
  }

  .positivo {
    color: green;
  }

  .negativo {
    color: red;
  }

  .mensaje {
    text-align: center;
    font-weight: bold;
    margin-top: 10px;
```

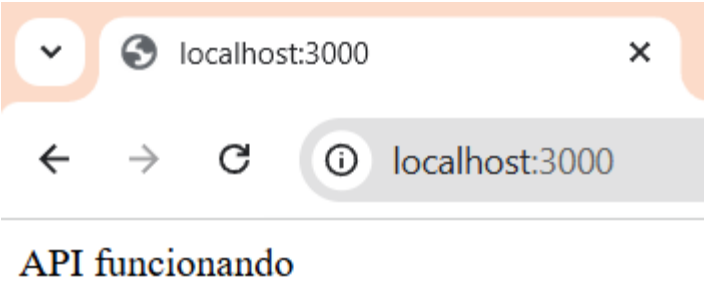
- **Node_modules/.env:**

```
.env
MONGO_URI=mongodb://kcarvajal1200_db_user:RosaEstela@ac-gs3wn4o-shard-00-00.d0yjiia.mongodb.net:27017,ac-gs3wn4o-shard-00-0
PORT=3000
```

Manejo de errores:

La API incluye manejo de errores para:

- **Datos inválidos (400 Bad Request)**
  Se presenta cuando los datos enviados no cumplen con las validaciones requeridas, como campos vacíos, valores incorrectos o formatos inválidos.
- **Transacción no encontrada (404 Not Found)**
  Ocurre cuando se intenta consultar, actualizar o eliminar una transacción que no existe en la base de datos.
- **Errores internos del servidor (500 Internal Server Error)**
  Se genera cuando ocurre un fallo inesperado en el servidor, como problemas en la conexión con la base de datos o errores en la ejecución del código.



- **Link YouTube: https://youtu.be/BFNAEuHgLqo**

Conclusión:

El desarrollo de este proyecto permitió aplicar de manera práctica los conocimientos adquiridos en programación web, especialmente en el uso de tecnologías como Node.js, Express y MongoDB. La implementación de una API REST y su integración con un frontend dinámico demuestra la importancia de la arquitectura cliente-servidor en el desarrollo de aplicaciones modernas.

Además, se logró construir una herramienta funcional que permite al usuario gestionar sus finanzas personales de manera eficiente. La incorporación de validaciones, filtros e indicadores financieros mejora la experiencia del usuario y garantiza la integridad de los datos, facilitando la toma de decisiones basada en información clara y organizada.

Finalmente, este proyecto evidencia el potencial de las aplicaciones web como soluciones prácticas a problemas cotidianos. Asimismo, sienta las bases para futuras mejoras, como la implementación de autenticación de usuarios, gráficos estadísticos o despliegue en la nube, lo que permitiría escalar el sistema a un nivel más profesional.